
Learning to Extrapolate in Generative Modeling

Preliminary draft. Please don't share or distribute.

Weichi Yao¹ Bryan Goldsmith² Yixin Wang³

Abstract

In recent years, conditional generative models have demonstrated remarkable success in producing realistic objects aligned with specified properties. However, these models often rely on abundant, representative datasets and struggle to accurately generate targets when the desired properties lie outside the observed training distribution. This challenge, commonly referred to as "extrapolation" in statistics, arises frequently in real-world generation tasks and is prevalent across scientific domains. Recognizing the importance of extrapolation under limited data conditions, this paper proposes learning an adapter that builds on top of the pretrained models. Augmenting these models with the learned adapter enables them to generate more accurate and reliable outputs for previously unseen target conditions. As a result, the adapted models demonstrate superior out-of-distribution performance, addressing a key limitation in practical generative modeling scenarios.

1. Introduction

Recent years have witnessed success of generative modeling success in scientific domains across biology, material sciences and etc.

With enough training data and model parameters, generative models are able to learn underlying structures to generate novel instances. However, these novel instances are the samples from *interpolating* of the training domain. Fundamentally, these generative models still face the challenge in their inherent extrapolation difficulty, that is, they are unable to *extrapolate*, i.e., to generate high quality of samples outside of training domain. In fact, the highly-capable non-

linear backbone architectures (GNNs, Transformers) that fit well in-range are often degrade sharply in the extrapolation region, while simpler model can be more stable when applied for out-of-range task.

Extrapolation is a statistical task that generalizes conclusions from observed data to novel unseen scenarios. In this paper, we aim to improve the extrapolative ability of conditional generative models. For example, in the context of molecule design, extrapolation aims to generate molecules that have labels outside the range of the training data labels. Such extrapolations guide the discovery and synthesis of molecules with optimized target properties.

Formally, let P_0 be a distribution over $(X, Y) \in \mathcal{X} \times \mathcal{Y}$, where X are input variables (features) and Y is a response variable (labels). We denote $\text{supp}(X)$ as the support of X , and $\text{supp}(Y)$ as the support of Y . Conditional generative modeling is interested in modeling the conditional distribution function $P(X | Y = y), y \in \mathcal{Y}$. Let $\mathcal{P}$ denote the family of distributions $P(\cdot | Y = y)$ for conditions $y \in \mathcal{Y}$. Extrapolation in conditional generative modeling refers to any statistical inference on

$$P(X | Y = y) \text{ for } y \in \mathcal{Y} \setminus \text{supp}(Y).$$

Extrapolation can typically only be meaningful given appropriate assumptions because the conditional distribution function $P(X | Y = y)$ is completely unconstrained by data for $y \in \mathcal{Y} \setminus \text{supp}(Y)$.

For a conditional generative model trained on samples whose labels $y \in \text{supp}(Y)$, and a target $y \in \mathcal{Y} \setminus \text{supp}(Y)$, the trained model most likely generates low-quality samples, whose underlying true labels deviate from the target y .

1.1. Main contribution

In this paper, we propose learning an adapter to augment pretrained models to improve extrapolative generation performance. Our main contributions include

- (1) We first present a general adapter for distributional extrapolation in the conditional generation $P(X | Y = y)$ setup in Section 2.

¹Michigan Institute for Data & AI in Society, University of Michigan ²Department of Chemical Engineering, University of Michigan ³Department of Statistics, University of Michigan. Correspondence to: Weichi Yao <weichiy@umich.edu>.

- (2) We introduce using transformers as an adapter for conditional flow matching in Section 3.
- (3) We demonstrate in Section 4 that in extrapolation tasks, by augmenting adapters, the pretrained generative models can generate more accurate and diverse outputs for previously unseen targets, substantially enhancing out-of-distribution performance.

1.2. Related works

Extrapolation under distribution shift has been studied primarily in supervised prediction. From a statistical learning perspective, extrapolation failures often arise because high-capacity models minimize empirical risk by exploiting shortcut features that are predictive on the training support but non-causal and non-transportable. Shimakawa et al. (2024) used a simple linear model with structured and constrained interactions on the derived quantum-mechanical descriptors to impose stronger inductive bias, which was shown to markedly improve extrapolative prediction performance over conventional nonlinear baselines under small experimental data regime.

Instead of restricting the model class that may undermines the model’s capacity to learn complex structure-to-property relationship, Noda et al. (2025) proposed extrapolative episodic training (E2T) that reframes prediction as support-set-conditioned inference and learns a meta-predictor $y = f_\phi(x, \mathcal{S})$ that predicts a query x given a labeled support set $\mathcal{S}$. Specifically, they trained it on episodes deliberately constructed so that the query and support come from different domains (e.g., held-out polymer classes or element combinations). Then a matching Neural Network in the form of resembling kernel ridge regression predicts by first computing similarities in a learned embedding space then doing a regularized, closed-form weighted combination of support labels.

For extrapolation under the generative modeling setup, Yao et al. (2024) proposed tail-extrapolative conditional molecule generation by adapting engression—pre-additive noise models—to generative modeling: instead of conditioning on a target property y directly, they condition on noisy copies $y + \eta$, yielding a distributional-extrapolation guarantee within a neighborhood. They showed that for both classifier-free guided $E(n)$ -equivariant diffusion on 3D molecules and ChemGPT-style conditional language models, by training with the modified objectives on the labeled and unlabeled data and sampling to average predictions across multiple noised conditions, the proposed method can improve tail property targeting and often increases diversity and uniqueness especially when the unlabeled data near the target is scarce.

In this work, instead of retraining the generative model

from scratch with noisy copies as in Yao et al. (2024), which can be computationally intensive, we leverage a strong pretrained conditional generative prior and learn a lightweight adapter that uses reference conditions to adjust the condition-to-distribution mapping. This is similar to the episodic intuition of E2T by Noda et al. (2025)—learning from query-reference relationships—but instead of operating over labeled support sets via attention-based aggregation, we leverage pretrained models and learn a generic adapter-conditioning pathway, yielding a more general and reusable conditioning procedure, and potentially reducing reliance on labeled target-domain support sets.

2. Adapter augmented conditional generative model

For any $y \in \mathcal{Y}$, let $f_{\bar{\theta}} : \mathcal{Y} \rightarrow \mathcal{X}$ denote a conditional generative model with parameters $\bar{\theta}$, pretrained on N training data points $(x_1, y_1), \dots, (x_N, y_N) \in \text{supp}(X) \times \text{supp}(Y)$ sampled from the true conditional distribution $f_0(y) := P_0(X | Y = y)$. Assume the generative model is well-trained on $\text{supp}(Y)$, i.e., for some small $\epsilon > 0$,

$$\sup_{y \in \text{supp}(Y)} \text{KL}(f_0(y) \| f_{\bar{\theta}}(y)) \leq \epsilon. \quad (1)$$

The extrapolation problem asks how to sample from $f_0(y^*)$ for $y^* \in \mathcal{Y} \setminus \text{supp}(Y)$, given we know how to sample from $f_0(y)$ through $f_{\bar{\theta}}(y)$ for $y \in \text{supp}(Y)$.

The intuition is that if such an extrapolation problem is ever solvable, it relies on the basic assumption that one can leverage the information from the learned domain $\text{supp}(Y)$, $(y, f_{\bar{\theta}}(y))$, to infer $f_0(y^*)$ for y^* from a non-overlapping unknown domain $\mathcal{Y} \setminus \text{supp}(Y)$. Formally, we present Assumption 2.1.

Assumption 2.1 (Existence). For any target $y^* \in \mathcal{Y} \setminus \text{supp}(Y)$, there exist (i) a set of *reference points* $\{y_1, \dots, y_n\} \subset \text{supp}(Y)$, selected by some *reference scheme* $R : \mathcal{Y} \rightarrow \mathcal{Y}^n$, and (ii) a mapping $h : (\mathcal{Y} \times \mathcal{X})^n \times \mathcal{Y} \rightarrow \mathcal{X}$, such that

$$h \circ f_0(y^*) := h\left((y_1, f_0(y_1)), \dots, (y_n, f_0(y_n)), y^*\right)$$

defines a distribution function *identical* to the true conditional distribution $f_0(y^*)$

$$h \circ f_0(y^*) \stackrel{d}{=} f_0(y^*), \quad \forall y \in \mathcal{Y}.$$

For $f_{\bar{\theta}}$ well-trained on $\text{supp}(Y)$ as in (1), Assumption 2.1 indicates $h(y^*) \stackrel{d}{\approx} f_0(y^*)$. Conceptually, the mapping h “knows” how to re-combine or re-weight the known distributions f_0 through $f_{\bar{\theta}}$ from reference group $R(y)$, to yield the correct distribution for a new query y .

We further introduce Assumption 2.2, where we assume the same R and h can work for all $y \in \mathcal{Y}$.

Assumption 2.2 (Universality). There exists a *universal reference selection* mechanism $R : \mathcal{Y} \rightarrow \mathcal{Y}^n$ and a *universal mapping* h such that for any target $y \in \mathcal{Y}$, one can select $(y'_1, \dots, y'_n) = R(y)$, and obtain a distribution from h that is identical to the true conditional distribution $f_0(y)$,

$$h \circ f_0(y) \stackrel{d}{=} f_0(y), \quad \forall y \in \mathcal{Y},$$

where $h \circ f_0(y)$ is defined w.r.t. target y 's reference set $R(y) = \{\tilde{y}_1, \dots, \tilde{y}_n\}$.

Under Assumption 2.2, such h is learnable by constructing training samples $(R(y), y)$ from the accessible region $\text{supp}(Y)$. Specifically, it indicates that, given a well-trained model $f_{\bar{\theta}}(y) \stackrel{d}{\approx} f_0(y)$ on $\text{supp}(Y)$, we have

$$h \circ f_{\bar{\theta}}(y) \stackrel{d}{\approx} f_{\bar{\theta}}(y), \quad y \in \text{supp}(Y). \quad (2)$$

The induced assumption on approximately identical distributions in (2) allows us to learn h_ϕ with model parameters ϕ by minimizing the KL divergence between $h_\phi \circ f_{\bar{\theta}}(y)$ and $f_{\bar{\theta}}(y)$ on $\text{supp}(Y)$,

$$\arg \min_{\phi} \sum_{y \in \text{supp}(Y)} \text{KL}(f_{\bar{\theta}}(y) \parallel h_\phi \circ f_{\bar{\theta}}(y)). \quad (3)$$

It also indicates that one can sample from $h \circ f_{\bar{\theta}}$, instead of directly from $f_{\bar{\theta}}$. In particular, for extrapolative response $y^* \notin \mathcal{Y} \setminus \text{supp}(Y)$, as long as $R(y^*) \subset \text{supp}(Y)$, one can sample from $f_0(y^*)$ approximately from $h_\phi \circ f_{\bar{\theta}}(y^*)$. We refer to such h as an *adapter* to the generative model $f_{\bar{\theta}}$.

Formally, we propose using the adapted generative model, $h_\phi \circ f_{\bar{\theta}}$ for conditional generation in the extrapolation region $\mathcal{Y} \setminus \text{supp}(Y)$. The adapter h_ϕ is trained in the same domain that $f_{\bar{\theta}}$ has access to. When $f_{\bar{\theta}}$ is well-trained over $\text{supp}(Y)$, we expect the train-to-learn-to-extrapolate $h_\phi \circ f_{\bar{\theta}}$ to be capable of generating more accurate samples compared to $f_{\bar{\theta}}$, at least within the neighboring region of $\text{supp}(Y)$, if not for all $\mathcal{Y} \setminus \text{supp}(Y)$.

3. Extrapolation via a Transformer-Based Adapter in Flow Matching

In what follows, we present our method to enable extrapolation in generative modeling by integrating a transformer-based adapter with a pretrained flow matching model.

Flow matching (Lipman et al., 2023) has recently emerged as a flexible framework for learning continuous-time generative processes, allowing one to gradually morph a simple noise distribution into a complex data distribution. However, like many generative techniques, it typically relies on

Algorithm 1 Training an adapter h (General)

Input: Training label support $\text{supp}(Y)$, a pretrained generative model p_θ , a reference selection algorithm R , number of reference points n , initial adapter parameters ϕ .

repeat

 Randomly sample $y' \in \text{supp}(Y)$.

 Select reference group $\{y_1, \dots, y_n\} \leftarrow R(y')$.

 Sample m data $x'_j \sim p_\theta(\cdot | y')$, $j = 1, \dots, m$.

for $i \in [1, 2, \dots, n]$ **do**

 Sample m data $x_{ik} \sim p_\theta(\cdot | y_i)$, $k = 1, \dots, m$.

end for

for $j \in [1, 2, \dots, m]$ **do**

$\hat{p}(x'_j | y') \leftarrow h_\phi(\{(y_i, x_{ik}, p_\theta(x_{ik} | y_i))\}_{ik}, y', x'_j)$

end for

 Compute loss $\ell(\phi) \leftarrow \sum_j \|\hat{p}(x'_j | y') - p_\theta(x'_j | y')\|$.

 Update model parameters ϕ .

until converge

Return: Trained adapter h_ϕ .

sampling within the range of observed training data, making extrapolation to out-of-distribution regions a challenge. To address this limitation, we introduce a transformer-based (Vaswani et al., 2017) adapter that leverages multi-head attention and feedforward layers to aggregate and reweight information from reference points, effectively guiding the flow matching process to generate samples for unseen or otherwise difficult target conditions.

Rather than altering the flow matching model itself, we place a transformer “on top,” allowing it to receive outputs from the flow matching model at various reference points. Internally, the transformer uses attention mechanisms to learn long-range dependencies in the conditioning space (e.g., different target values of y), and outputs a refined or adapted update that better accounts for out-of-distribution behavior.

In the following sections, we first describe the fundamentals of flow matching, then present the design and training algorithm of our transformer-based adapter that enables robust extrapolation performance.

Flow matching is a new paradigm for generative modeling that targets a probabilistic density path p_t that flows from some simple distribution p_{noise} to the target data distribution p_{data} . The idea is to construct and learn a Continuous Normalizing Flow (CNF), ϕ_t , to match the target probability path p_t . The probabilistic path p_t is generated by the vector field u_t . Similarly, one can parametrize a vector field v_θ to construct the CNF ϕ_t via the ordinary differential equation

$$\frac{d}{dt} \phi_t(z) = v_\theta(\phi_t(z)), \quad \phi_0(z) = z.$$

During the training, flow matching regresses the vector field u_t with the CNF model v_θ . The training objective for condi-

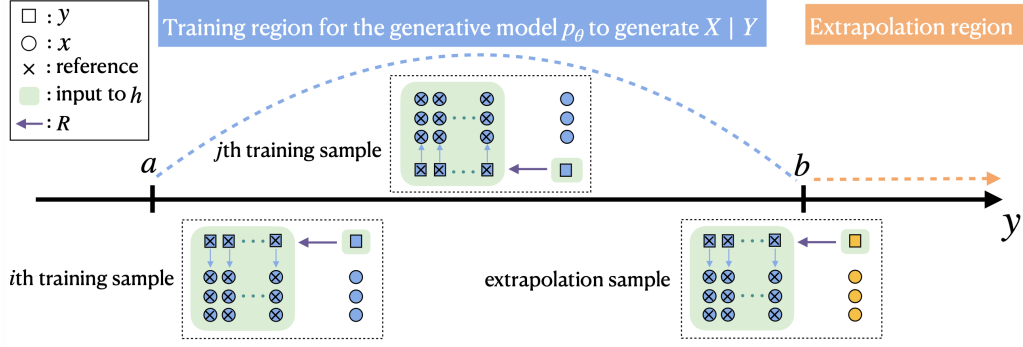


Figure 1. Data examples for training and evaluation in extrapolation tasks on one-dimensional y . The support $\text{supp}(Y)$ of the training data for the generative model p_θ is (a, b) , and the target extrapolation region is $(b, +\infty)$. The goal is to sample from $y' \in (b, +\infty)$. The training data for the adapter h is constructed within (a, b) , see the two circled dashed boxes. In each of these dashed boxes, (i) the target y' (squares), randomly sampled in (a, b) , (ii) the reference samples y 's (marked squares), selected by R (purple arrows) for the target y , and (iii) three reference samples x (marked circles) drawn from p_θ , are used as inputs to the adapter h to approximate the generation of x (circles) for target condition y' . The solid box circles out an evaluation data example illustrating the extrapolation for $y' \in (b, \infty)$, —generating the target (circles)— by passing the target y' (squares), the reference y 's (marked squares), and their sampled x 's (marked circles), into the adapter h .

tional generation task is then defined in the form of

$$L_{\text{FM}}(\theta) = \mathbb{E}_{t, z \sim p_t} \|u_t(z) - v_\theta(z, t)\|^2. \quad (4)$$

4. Empirical studies

4.1. Synthetic datasets

In this section, two synthetic datasets (x, y) where $x \in \mathbb{R}^d$ is generated from $y \in \mathbb{R}$ are constructed for the conditional generation task. For the first synthetic dataset, we consider a linear mapping from three square terms of y to x ,

$$(S1) \quad x = y^2 w_1 + (y - 2)^2 w_2 + (y + 3)^2 w_3,$$

where $w_i \in \mathbb{R}^d$, $i = 1, 2, 3$, are prespecified parameters randomly generated from $\mathcal{N}(0, I)$. For the second synthetic dataset, $x \in \mathbb{R}^d$ is generated by computing the sum of a sinusoidal term and a logarithmic term of y ,

$$(S2) \quad x = c_0 + c_1 \cdot \sin(y + c_2) + c_4 \cdot \log((y + 1)^2 + c_3),$$

where $c_0, c_1, c_2, c_3, c_4 \in \mathbb{R}^d$ are prespecified parameters randomly drawn from distributions $\mathcal{U}(0.5, 3.0)^d$, $\mathcal{U}(0.5, 2.0)^d$, $\mathcal{U}(-\pi, \pi)^d$, $\mathcal{U}(-0.5, 0.5)^d$, and $\mathcal{U}(0.5, 1.5)^d$, respectively.

We generate the datasets $\mathcal{D}_1$ and $\mathcal{D}_2$, accessible to the generative model, for synthetic setup (S1) and (S2), respectively. Specifically, we set $\mathcal{D}_1 = \{(x, y) : y \in \mathcal{Y}_1\}$ with $\mathcal{Y}_1 = [-30, 24]$ and $\mathcal{D}_2 = \{(x, y) : y \in \mathcal{Y}_2\}$ with $\mathcal{Y}_2 = [10, 70]$. We consider the extrapolation region $\check{\mathcal{Y}}_1 = (24, +\infty)$ and $\check{\mathcal{Y}}_2 = (70, +\infty)$ for each setup.

Under each setup, a flow matching model f is trained on $\mathcal{D}$ to minimize the loss in (4). This pretrained generative model

f can be used to generate x for any given y . An adapter h is then trained on the reference-pair datasets $\mathcal{R}$. Each data sample in $\mathcal{R}$ is in the form of $\{(x, y), (x_1, y_1), \dots, (x_n, y_n)\}$, where $y, y_1, \dots, y_n \in \mathcal{D}$, and the corresponding x are generated using the pretrained model f_θ . We evaluate the performance of both f and $h \circ f$ on the extrapolation region $\check{\mathcal{Y}}$ by examining the gap between the target response value and its true response value of the generated x , $y_{\text{target}} - y_{\text{true}}$.

Figure 2 presents the results for datasets generated under setup S1 (left) and S2 (right). In both cases, the generic flow matching model f effectively generates x such that the true response values closely align with the target values ($y_{\text{target}} - y_{\text{true}} \approx 0$) within the training regions $\mathcal{Y}$ (see $y_{\text{target}} \in \{19, 20, 21, 22, 23, 24\}$ in setup S1, $y_{\text{target}} \in \{68, 69, 70\}$ in setup S2). By contrast, the Transformer-adapted flow matching model $h \circ f$ shows a slight decrease in in-distribution performance. This can be due to the fact that the additional flexibility of the adapter h may nudge some predictions away from the in-distribution optimum in order to better accommodate challenging or unseen regions. Moreover, this added complexity can induce a mild mismatch in objectives, since the base model f focuses on fitting the training manifold while the adapter h learns to reconcile both in-distribution fidelity and extrapolative robustness.

For the extrapolation regions $\check{\mathcal{Y}}$ ($\check{\mathcal{Y}}_1 = (24, +\infty)$ for setup S1, and $\check{\mathcal{Y}}_2 = (70, +\infty)$ for setup S2), the true response values y_{true} of the generated x from both models tend to be always smaller than the target response values y_{target} , which is expected as the corresponding training region $\mathcal{Y}$ is to the left of the corresponding extrapolation region. However, as y_{target} moves further into $\check{\mathcal{Y}}$ (see y_{target} going from

24.5 to 29 for setup S1, and y_{target} going from 70.5 to 75 for setup S2), the discrepancy increases, much less catastrophically when x are generated from $h \circ f$ than those from f . It shows that, if we train a Transformer-based adapter h on top of the generic flow matching model f , the resulted Transformer-adapted flow matching model $h \circ f$ is able to enhance generalization beyond the training region and improve extrapolative robustness over the generic flow matching model f .

4.2. QM9 dataset

The inverse design of molecules is a quintessential example of extrapolation tasks in conditional generative modeling. As one is often interested in designing molecules with exceptional properties, the support of the response variable $\text{supp}(Y)$ often occupies some middle range of values, and the region of extrapolation $\mathcal{Y} \setminus \text{supp}(Y)$ often lies on the tail, e.g. some target properties take either extremely small values or extremely large values.

We use QM9 (Ramakrishnan et al., 2014) to evaluate the extrapolative performance for the property HOMO-LUMO Gap (Hartree) between the generic (f) and extrapolation-adapted generative models ($h \circ f$). QM9 contains molecular properties and atom coordinates for 130K small molecules with up to 29 atoms (of type H, C, N, O, F). To study the generative model performance, we split the dataset w.r.t. the property HOMO-LUMO Gap (y) of the molecule. To ensure the accuracy level of HOMO-LUMO Gap between the molecules in the training dataset and the newly generated molecules, we recompute the HOMO-LUMO Gap in the training dataset using PSI4 python package, and evaluate the generated molecules using the same package. The base flow matching model f is trained on the molecules whose HOMO-LUMO Gap y (by PSI4) all fall into the region $(-\infty, 0.33)$, and the extrapolative task is defined as generating molecules with $y \geq 0.33$. During the generation, f and $h \circ f$ are to set to generate molecules x given y_{target} . To evaluate the generative performance, we first check the validity of the molecules, and then compute the true property value of the generated molecules using PSI4, and its distance from targeted property value $|y - y_{\text{target}}|$ (the smaller the better).

Figure 3 shows the generative performance of both generative model f and the proposed extrapolative adapter $h \circ f$ in the near- (0.28, 0.32) and in-extrapolative region (0.34, 0.42). The generative model f outperforms the extrapolative adapter $h \circ f$ within the training domain when $y_{\text{target}} < 0.33$, as expected. As the target property value y moves further away from the training domain, f often fails to generate any valid molecules (see $y_{\text{target}} \in (0.37, 0.39)$) and $y \in (0.41, 0.43)$ even when it does, the deviation from the targeted property value $|y - y_{\text{target}}|$ is large. Compar-

atively, $h \circ f$ is able to generate samples that are valid molecules most of the times, and the deviation is much smaller.

5. Conclusions

It is fairly common to see a slight drop in in-distribution performance when layering an adapter on a well-tuned base model—particularly if that adapter is designed to enhance generalization or extrapolation beyond the training region, thereby trading off a measure of local precision for improved coverage of out-of-distribution conditions. Specifically, the adapter’s additional flexibility may nudge some predictions away from the in-distribution optimum in order to better accommodate challenging or unseen regions. Moreover, this added complexity can induce a mild mismatch in objectives, since the base model focuses on fitting the training manifold while the adapter learns to reconcile both in-distribution fidelity and extrapolative robustness.

Software and Data

References

- Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=PqvMRDCJT9t>.
- Noda, K., Wakiuchi, A., Hayashi, Y., and Yoshida, R. Advancing extrapolative predictions of material properties through learning to learn using extrapolative episodic training. *Communications Materials*, 6, 2025.
- Ramakrishnan, R., Dral, P. O., Rupp, M., and Von Lilienfeld, O. A. Quantum chemistry structures and properties of 134 kilo molecules. *Scientific data*, 1:1–7, 2014.
- Shimakawa, H., Kumada, A., and Sato, M. Extrapolative prediction of small-data molecular property using quantum mechanics-assisted machine learning. *npj Computational Materials*, 10, 2024.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. u., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Yao, W., Gruich, C., Goldsmith, B., and Wang, Y. Tail extrapolation in target-aware conditional molecule generation. In *ICML 2024 AI for Science Workshop*, 2024.

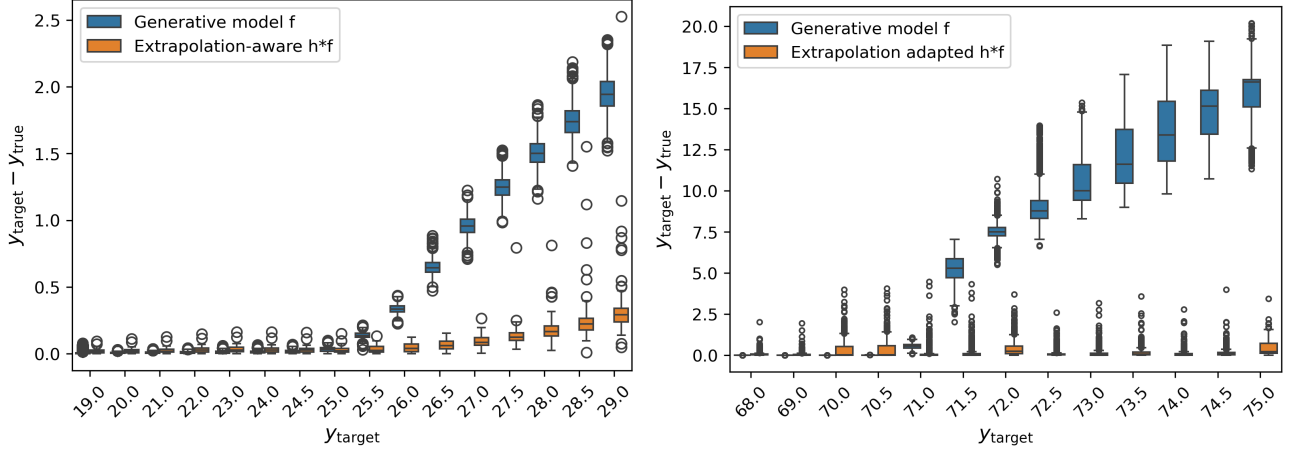


Figure 2. Boxplots illustrating the difference between the target response value and the true response value, $y_{\text{target}} - y_{\text{true}}$, for generated x targeting y_{target} using the generic flow matching model f and the Transformer-based adapted flow matching model $h \circ f$. The left plot corresponds to the synthetic dataset $\mathcal{D}_1$, where $\mathcal{Y}_1 = (-30, 24]$, while the right plot corresponds to the synthetic dataset $\mathcal{D}_2$, where $\mathcal{Y}_2 = (10, 70]$. At each value of y_{target} on the x -axis, the pair of boxplots represents results from f (blue, left) and $h \circ f$ (orange, right). In both cases, the generic flow matching model f accurately generates x such that the true response values closely match the target values ($y_{\text{target}} - y_{\text{true}} \approx 0$) when $y_{\text{target}} \in \mathcal{Y}_1$ or $\mathcal{Y}_2$. However, for $y_{\text{target}} \in \check{\mathcal{Y}}_1 = (24, +\infty)$ or $\check{\mathcal{Y}}_2 = (70, +\infty)$, deviations increase, with larger discrepancies occurring as y_{target} moves further into $\check{\mathcal{Y}}_1$ or $\check{\mathcal{Y}}_2$.

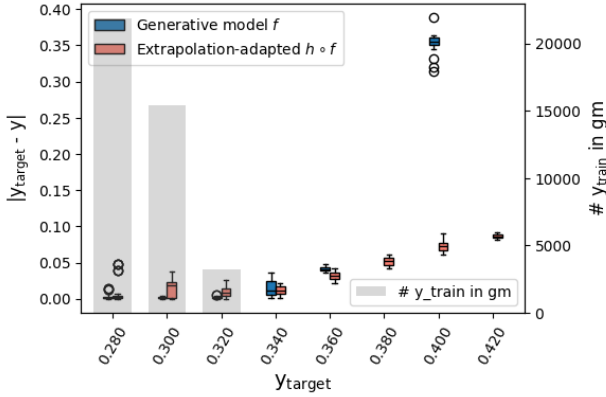


Figure 3. Boxplots illustrating the difference between the target property value and the true property value, $y_{\text{target}} - y$, for generated x targeting y_{target} using the generic flow matching model f and the Transformer-based adapted flow matching model $h \circ f$ on QM9 dataset. At each value of y_{target} on the x -axis, the pair of boxplots represents results from f (blue, left) and $h \circ f$ (orange, right). In both cases, the generic flow matching model f accurately generates x such that the true property values closely match the target values ($y_{\text{target}} - y \approx 0$) within the training domain $y_{\text{target}} < 0.33$. However, for $y_{\text{target}} \geq 0.33$, deviations increase, with larger discrepancies occurring as y_{target} becomes larger.